

SOFTWARE ARCHITECTURE REVIEW

NovaFacts Production Readiness Audit

Independent Second-Pass Review

DATE

2026-06-25

SCOPE

project-backend · project-frontend ·
project-docs

STACK

Spring Boot 3.5 · Vue 3 · PostgreSQL

REVISION

Second Pass (v2)

CONTEXT

Ingeniería de Software 1 — UNAL

OVERALL SCORE

18 / 100

Executive Summary

Overall project health: Academic prototype, pre-MVP. One feature slice (`auth`) is implemented end-to-end but is broken at the integration seam — the login button produces a `console.log` and no navigation. The domain core (bookings, invoicing, penalties, billing — the actual stated purpose of the system) exists only as disconnected POJOs with no controller, persistence, or API surface.

Production readiness: Not production ready. There is no deployable, internally-consistent vertical slice you could put in front of real users today. The one user-facing flow that exists (login) is functionally broken end-to-end when run through the documented setup.

Progress since prior audit (2026-06-24): Three issues were resolved — the Docker container-to-container DB port was corrected to 5432, the `LoginForm.vue / authService.js` duplicate login implementation was removed, and CORS was centralized into a global `CorsConfigurationSource` bean. All other prior-audit findings remain open. Twelve new issues were identified in this pass.

Dimension	Score	Rationale
Security	10/100	No token issued; all endpoints <code>permitAll()</code> ; unauthenticated deletion; hardcoded DB credentials in git
Authentication	5/100	BCrypt password hashing is correct; nothing else — no token, no session, no auth state
Authorization	0/100	No authorization model exists at all
Completeness	10/100	Login doesn't navigate on success; booking/invoicing domain entirely unimplemented as API
Backend Architecture	45/100	Clean layer separation in <code>auth/</code> ; constructor injection; but no <code>@Valid</code> , no exception handler, no logging, wrong HTTP status codes
Frontend Architecture	30/100	Clean view/component split; TypeScript partially adopted; no proxy; no env vars; dead routes; no post-login navigation
Infrastructure	35/100	Docker Compose mostly correct (DB port fixed); Dockerfile EXPOSE wrong; hardcoded secrets; no profiles; no migration tool
Testing	40/100	Good unit coverage for domain classes; zero integration tests; context test is a no-op
Observability	10/100	No application logging; TRACE SQL is the only signal; no real health endpoint
Documentation	40/100	<code>skill_*.md</code> self-audits are good but now lag the code; README at docs level is solid

Status of Prior Audit Findings

Prior Issue	Status	Evidence
#3 — Docker network DB URL used host port 5434	✓ FIXED	<code>docker-compose.yml:30</code> now uses <code>postgres_db:5432</code>
#6 — Duplicate login implementations (<code>LoginForm.vue</code> + <code>authService.js</code>)	⚠ PARTIAL	Both files deleted. However <code>SecretView.vue</code> , <code>SecretDisplay.vue</code> , and <code>/secret</code> route remain, and <code>LoginView.vue</code> still does not navigate on login success.
#8 — CORS not configured for <code>/api/auth/login</code>	✓ FIXED	<code>SecurityConfig.java:34-43</code> registers a <code>CorsConfigurationSource</code> bean covering <code>/api/**</code>
All other prior findings (#1, #2, #4, #5, #7, #9-#25)	✗ STILL OPEN	Verified individually by reading current source files

Critical Severity Findings

CRITICAL

C-01

No real authentication is ever issued — login is a functional dead end

Category	Security / Authentication
Files	<code>auth/dto/LoginResponse.java</code> · <code>auth/service/UserService.java:76</code> · <code>src/services/authService.ts:17</code> · <code>src/views/LoginView.vue:113-128</code>
Status	From prior audit · Still present

TECHNICAL EXPLANATION

`UserService.login()` returns `new LoginResponse("Login exitoso")` — a plain struct with one field, `message`. No token, session ID, or credential artifact of any kind is issued. There is no JWT dependency, filter, or session mechanism in the backend (`pom.xml` has no `jjwt`, `OAuth2`, or equivalent; `com.novafacts.backend.security` is an empty directory). `authService.ts:17` reads `response.data.token` which will always be `undefined` since the backend never sends that field. `LoginView.vue:127-128` handles the success case with a `console.log` and a commented-out `router.push` — there is no navigation, no token storage, and no way for any subsequent request to carry authentication proof.

IMPACT

The only user-facing feature (authentication) does not actually authenticate anything. A successful login produces no session, navigates nowhere, and leaves the user on the login screen.

RECOMMENDED SOLUTION

Implement stateless JWT: `LoginResponse` must include a `token` field; add a Spring Security JWT filter for validation; store the token on the frontend; call `router.push()` to a protected route on success.

RISK OF FIX

High — requires coordinated changes across both repos. This is the anchor for all authorization work.

CRITICAL

C-02

Unauthenticated user enumeration and arbitrary account deletion

Category Security / Authorization**Files** `auth/controller/UserController.java:27-35` ·
`config/SecurityConfig.java:26`**Status** **From prior audit · Still present**

TECHNICAL EXPLANATION

`SecurityConfig` applies `.anyRequest().permitAll()` with no exceptions. `UserController` exposes `GET /api/users` (returns every user's `id` and `username`, no limit, no auth check) and `DELETE /api/users/{id}` (deletes any account by ID, no auth check, no ownership verification). If the target ID does not exist, `deleteById()` throws `EmptyResultDataAccessException`, which surfaces as an opaque 500.

IMPACT

Any anonymous HTTP client can enumerate all usernames and delete any account — including future admin accounts — with a single HTTP request.

RECOMMENDED SOLUTION

Block behind real authentication (C-01) and role-based access control. Restrict `DELETE` to admin roles. Add `@ResponseStatus(HttpStatus.NO_CONTENT)`.

RISK OF FIX

Medium once C-01 is implemented.

CRITICAL

C-03

Hardcoded database credentials committed to version control**Category** Security / Secrets Management**Files** `src/main/resources/application.properties:5` · `docker-compose.yml:11, 32`**Status** **From prior audit · Still present****TECHNICAL EXPLANATION**

`application.properties:5` contains `spring.datasource.password=Epsilon_Delta.0` in plaintext with no environment variable indirection. `docker-compose.yml:11, 32` also hardcodes the same value. Both files are git-tracked. The project's own prior self-audit (`skill_back.md` , row S4) falsely claimed configuration was externalized.

```
application.properties:5  spring.datasource.password=Epsilon_Delta.0
docker-compose.yml:11     POSTGRES_PASSWORD: Epsilon_Delta.0
docker-compose.yml:32     SPRING_DATASOURCE_PASSWORD=Epsilon_Delta.0
```

IMPACT

Any contributor with read access to the repository has the database password. The credential is in git history and cannot be removed without a full history rewrite.

RECOMMENDED SOLUTION

Replace values with `${POSTGRES_PASSWORD}` placeholders. Add `.env` to `.gitignore` . Provide `.env.example` . Rotate the exposed password before any real deployment.

RISK OF FIX

Low mechanically; medium for coordinating secret rotation in team workflows.

CRITICAL

C-04

Core business domain has no API, entity, or persistence layer**Category**

Architecture / Completeness

Files

`booking/model/Booking.java` ·
`booking/service/BookingValidator.java` ·
`booking/service/InvoiceCalculator.java` · `Esquema_BD.sql`

Status**From prior audit · Still present****TECHNICAL EXPLANATION**

NovaFacts' stated purpose is financial management for short-term-rental bookings (anticipos, penalidades, facturación, devoluciones). The entire domain implementation consists of three plain Java classes with no `@Entity`, no repository, and no controller. `Esquema_BD.sql` defines 14 tables — none of which has a corresponding JPA entity. The `booking/` package has no `entity/`, `repository/`, or `controller/` subpackage.

IMPACT

The product's advertised purpose is entirely unimplemented from an API perspective. What exists is a unit-tested rule engine with no way to reach it over HTTP or persist its results.

RECOMMENDED SOLUTION

Treat as primary development phase, not a bug fix. Introduce Flyway migrations derived from `Esquema_BD.sql` before building any entity layer.

RISK OF FIX

High — requires significant new development; no risk of regression in existing code.

High Severity Findings

HIGH

H-01

Login error messages leak whether a username exists

Category Security / Information Disclosure**Files** `auth/service/UserService.java:65, 72-74`**Status** **From prior audit · Still present**

`UserService.login()` throws `"Usuario no encontrado"` when the username doesn't exist, and `"Contraseña incorrecta"` when the username exists but the password is wrong. Both propagate as uncaught `RuntimeException` with Spring's default error body, preserving the distinct messages. Combined with the unauthenticated `GET /api/users` (C-02), this is a low-effort enumeration vector.

RECOMMENDED SOLUTION

Return a single generic error — `"Credenciales inválidas"` — for both failure cases. Map both to `401 Unauthorized` in the global exception handler (H-03).

HIGH

H-02

Dead configuration classes outside Spring component-scan boundary

Category Architecture / Dead Code**Files** `com/novafacts/SecurityConfig.java` ·
`com/novafacts/config/PasswordConfig.java`**Status** **From prior audit · Still present**

`@SpringBootApplication` in `com.novafacts.backend` scans only `com.novafacts.backend.*`. The dead `SecurityConfig` and `PasswordConfig` live in `com.novafacts` and `com.novafacts.config` — sibling packages never scanned. Neither class is ever registered as a bean. The dead `PasswordConfig` also declares a `PasswordEncoder` bean that, if accidentally activated, would cause an ambiguous-bean startup failure.

RECOMMENDED SOLUTION

Delete both files. Zero risk — they are dead code that has never been loaded.

HIGH

H-03

No global exception handler — all domain errors surface as 500s

Category Backend / Error Handling

Files `auth/service/UserService.java:65,73` ·
`auth/controller/AuthController.java` ·
`auth/controller/UserController.java`

Status **From prior audit · Still present**

There is no `@ControllerAdvice` or `@ExceptionHandler` anywhere in the codebase (verified by repo-wide grep). `UserService` throws raw `RuntimeException` for all domain failures. Spring converts any uncaught runtime exception to HTTP `500 Internal Server Error`. Clients cannot distinguish "bad credentials" from "server crashed" by status code alone.

RECOMMENDED SOLUTION

Add a `@ControllerAdvice` in `com.novafacts.backend.common`. Map domain exceptions: `AuthenticationException` → 401, entity not-found → 404, duplicate key → 409, `IllegalArgumentException` → 400.

HIGH

H-04

No request validation anywhere in the API

Category Security / Robustness

Files `auth/dto/CreateUserRequest.java` · `dto/LoginRequest.java` ·
`pom.xml`

Status **From prior audit · Still present**

No DTO has any `jakarta.validation` constraint. No controller method has `@Valid`. `pom.xml` does not include `spring-boot-starter-validation`. A `POST /api/users` with `{"username":"","password":""}` will succeed — `passwordEncoder.encode("")` produces a valid BCrypt hash of an empty string. The `@Column(nullable = false)` on `User.username` permits empty strings (it only blocks SQL `NULL`). Additionally, `User.username` has no `unique = true`, so duplicate usernames are silently accepted (see H-08).

RECOMMENDED SOLUTION

Add `spring-boot-starter-validation`; annotate DTOs with `@NotBlank`, `@Size`; add `@Valid` to controller parameters.

HIGH

NEW

H-05

LoginView.vue completes login but performs no navigation

Category Frontend / Broken Feature

Files `src/views/LoginView.vue:113-128`

Status Newly identified

`handleFormSubmit()` handles error cases correctly but on `status === 'success'` executes only `console.log('Login exitoso para el token:', result.token)`. The redirect is left as a comment. `useRouter()` is not even imported. `result.token` is always `undefined` at runtime (backend never sends a `token` field). The user remains on the login page regardless of whether authentication succeeded. Login is entirely non-functional from the user's perspective.

RECOMMENDED SOLUTION

Blocked by C-01. Once the backend issues a real token, import `useRouter`, store the token, and call `router.push('/dashboard')`.

HIGH

NEW

H-06

SecretView.vue, SecretDisplay.vue, and /secret route are orphaned dead code

Category Frontend / Dead Code

Files `src/views/SecretView.vue` · `src/components/SecretDisplay.vue` · `src/router/index.js:14-17`

Status Newly identified

Since `LoginForm.vue` was deleted and `LoginView.vue` never calls `router.push()` (H-05), there is no code path that navigates to `/secret`. `SecretView.vue`, `SecretDisplay.vue`, and the `/secret/:secretPhrase?` route are unreachable from the live application flow. Additionally, the route declares `:secretPhrase` as a path parameter, but `SecretView.vue:23` reads `route.query.phrase` — a query string parameter — so the component was never tested end-to-end.

RECOMMENDED SOLUTION

Delete `SecretDisplay.vue`, `SecretView.vue`, and the `/secret` route unless the feature is actively being built. Zero risk of regression.

HIGH

NEW

H-07

HTTP Basic auth enabled alongside permitAll() — contradictory security posture

Category Security / Configuration

Files config/SecurityConfig.java:28

Status Newly identified

`SecurityConfig` registers `.httpBasic(Customizer.withDefaults())`, installing Spring Security's `BasicAuthenticationFilter` in the chain, while simultaneously applying `.anyRequest().permitAll()`. The filter processes credentials but never gates access. When the authorization policy is eventually tightened, Spring will start prompting for HTTP Basic credentials in the browser rather than returning a clean JSON `401` that the SPA can handle. Wrong default for a REST API destined for JWT.

RECOMMENDED SOLUTION

Remove `.httpBasic()`. Add `.sessionManagement(sm -> sm.sessionCreationPolicy(STATELESS))`. No risk — removing it has no effect given `permitAll()`.

HIGH

NEW

H-08

username field has no uniqueness constraint — duplicate accounts can be created

Category Backend / Data Integrity

Files auth/entity/User.java:13-14

Status Newly identified

`User.username` is `@Column(nullable = false)` with no `unique = true`. No `@Table(uniqueConstraints = ...)` is present. Two or more users can be created with the same username. `UserRepository.findByUsername()` returns `optional<User>` — if multiple rows match, Hibernate throws `IncorrectResultSizeDataAccessException`, producing a 500 with no useful message. Login is then broken for any duplicated username.

RECOMMENDED SOLUTION

Add `@Column(nullable = false, unique = true)` to `User.username`. Add a service-level uniqueness check returning `409 Conflict` before the DB constraint fires.

Medium Severity Findings

MEDIUM

M-01

Money modeled as double in financial calculations

Category Code Quality / Correctness

Files booking/model/Booking.java:10 ·
booking/service/InvoiceCalculator.java:7-9 ·
Esquema_BD.sql:50, 65, 86

Status From prior audit · Still present

pricePerNight and all derived invoice arithmetic use double . Esquema_BD.sql correctly types monetary columns as decimal(12,2) . IEEE-754 binary floating point cannot represent most decimal fractions exactly, introducing rounding error that compounds through tax and discount calculations — a correctness risk for a billing system handling Colombian Peso amounts.

RECOMMENDED SOLUTION

Replace double with BigDecimal for all monetary fields and arithmetic. Use RoundingMode.HALF_UP at each step. Align with the decimal(12,2) schema.

MEDIUM

M-02

No Spring Profiles — verbose SQL logging is unconditional

Category Infrastructure / Observability

Files src/main/resources/application.properties:7, 10-11

Status From prior audit · Still present

A single application.properties unconditionally enables spring.jpa.show-sql=true , logging.level.org.hibernate.SQL=DEBUG , and logging.level.org.hibernate.type.descriptor.sql.BasicBinder=TRACE . In production, TRACE-level logging of SQL binder parameters exposes all bound values (including passwords passed to queries) in log files.

RECOMMENDED SOLUTION

Create application-dev.properties with verbose logging. Use SPRING_PROFILES_ACTIVE=prod in docker-compose.yml to suppress it in the containerized stack.

MEDIUM

M-03

ddl-auto=update with no migration tool; Esquema_BD.sql is orphaned**Category** Infrastructure / Data Management**Files** `application.properties:8` · `docker-compose.yml` · `Esquema_BD.sql`**Status** **From prior audit · Still present**

Schema evolution is managed by Hibernate's `update` strategy. `Esquema_BD.sql` (229 lines, 14 tables) is never executed anywhere — no Flyway/Liquibase dependency in `pom.xml`, no init-script mount in the postgres service. The one entity that exists (`User`) maps to a `users` table whose schema does not match the `usuario` table in `Esquema_BD.sql`. Schema drift is invisible and uncontrolled.

RECOMMENDED SOLUTION

Add Flyway. Convert `Esquema_BD.sql` (after reconciling the `usuario` vs. `User` discrepancy) to the first migration. Switch `ddl-auto` to `validate` or `none`.

MEDIUM

M-04

Dockerfile declares EXPOSE 8081; application listens on 8082**Category** Infrastructure / Configuration**Files** `Dockerfile:4` · `application.properties:13` · `docker-compose.yml:28`**Status** **From prior audit · Still present**

`Dockerfile:4` declares `EXPOSE 8081`. The app sets `server.port=8082` and `docker-compose.yml` maps `"8082:8082"`. `EXPOSE` is non-binding metadata, so the compose port mapping works — but anyone running the image standalone with `docker run -p 8081:8081 ...` gets no response.

RECOMMENDED SOLUTION

Change `EXPOSE 8081` to `EXPOSE 8082`. No risk.

MEDIUM

M-05

Frontend API URL relies on relative path with no proxy configured

Category Frontend / Configuration

Files `src/services/authService.ts:10` · `vite.config.js`

Status **From prior audit · Still present**

`authService.ts:10` calls `axios.post('/api/auth/login', ...)` with a relative URL and no `baseUrl` on the Axios instance. `vite.config.js` has no `server.proxy` configuration. In development, the request goes to the Vite dev server (`localhost:5173/api/auth/login`), which returns 404. No `.env` or `.env.development` file exists. The login endpoint is effectively unreachable in the documented local dev workflow.

RECOMMENDED SOLUTION

Add `server.proxy: { '/api': { target: 'http://localhost:8082', changeOrigin: true } }` to `vite.config.js`. Add `.env` files with `VITE_API_BASE_URL` for non-proxy environments.

MEDIUM

M-06

No router-level navigation guards — route protection is inside a component

Category Frontend / Security Architecture

Files `src/router/index.js` · `src/views/SecretView.vue:20-31`

Status **From prior audit · Still present**

No `router.beforeEach()` hook and no `meta.requiresAuth` field exist. The only "protection" for `/secret` is a check inside `SecretView.vue`'s `onMounted` for a query parameter. This is not authentication — it can be bypassed by navigating to `/secret?phrase=anything`. Sets a precedent for security logic in leaf components rather than the routing layer.

RECOMMENDED SOLUTION

Add `router.beforeEach()` keyed on real auth state (token in Pinia store). Apply `meta: { requiresAuth: true }` to all protected routes. Blocked by C-01.

MEDIUM

M-07

getUsers() loads the entire table with no pagination**Category** Backend / Scalability**Files** auth/service/UserService.java:51-60**Status** **From prior audit · Still present**

`getUsers()` calls `userRepository.findAll()` — a `SELECT * FROM users` with no `LIMIT` or `OFFSET`. As the table grows, every row is loaded into memory and serialized. Combined with C-02 (unauthenticated), an anonymous caller can force full table serialization on every request.

RECOMMENDED SOLUTION

Replace with `userRepository.findAll(Pageable pageable)` and accept page/size query parameters in the controller.

MEDIUM**NEW**

M-08

No application-level logging in service classes**Category** Observability**Files** auth/service/UserService.java**Status** **Newly identified**

Despite Lombok being on the classpath, no service class uses `@Slf4j` or declares a `Logger`. `UserService` performs user creation, login attempts, and deletion with zero log statements. The only signal available is Hibernate SQL at `DEBUG / TRACE`. For a financial management system, login events, account creation, and account deletion should produce audit log entries.

RECOMMENDED SOLUTION

Add `@Slf4j` to service classes. Log at `INFO` for domain events; `WARN / ERROR` for unexpected conditions. Purely additive — no risk.

MEDIUM

NEW

M-09

REST HTTP status codes are semantically wrong on mutation endpoints**Category** Backend / API Design**Files** auth/controller/UserController.java:20-35**Status** Newly identified

`createUser()` returns `UserResponse` with Spring's default `200 OK`. REST convention for resource creation is `201 Created`. `deleteUser()` returns `void`, which Spring maps to `200 OK` with an empty body. REST convention for deletion is `204 No Content`. Neither controller method has a `@ResponseStatus` annotation.

RECOMMENDED SOLUTION

Add `@ResponseStatus(HttpStatus.CREATED)` to `createUser()` and `@ResponseStatus(HttpStatus.NO_CONTENT)` to `deleteUser()`. No risk.

Low Severity Findings

ID	Title	Category	Status
L-01	Scaffold controllers (<code>TestController</code> , <code>HelloController</code>) deployed in production source	Dead Code	Prior audit · Open
L-02	<code>HelloWorld.vue</code> scaffold component and unused assets (<code>vite.svg</code> , <code>vue.svg</code> , <code>hero.png</code> , <code>style.css</code>) still present	Dead Code	Prior audit · Open
L-03	Empty <code>common/</code> and <code>security/</code> packages — scaffolded but never filled	Architecture	Prior audit · Open
L-04	Orphaned <code>package-lock.json</code> files at workspace root and <code>project-frontend/</code> root (no companion <code>package.json</code>)	Repository Hygiene	Prior audit · Open
L-05	Stale <code>backend/</code> directory at workspace root (only <code>.idea/</code> and <code>.gitignore</code>)	Repository Hygiene	Prior audit · Open
L-06 NEW	"Forgot password" link is a dead <code></code> — no implementation exists	UX / Completeness	Newly identified
L-07 NEW	<code>authService.ts</code> email validation is dangerously weak (<code>includes('@')</code>)	Frontend / Validation	Newly identified
L-08 NEW	"Remember me" feature stores only the email in <code>localStorage</code> — no session state, value never read back	Incomplete Feature	Newly identified
L-09 NEW	Frontend is a mixed JS/TS codebase with no <code>tsconfig.json</code> — TypeScript migration is incomplete	Code Quality	Newly identified
L-10 NEW	<code>pom.xml</code> metadata fields are all empty stubs (<code><name/></code> , <code><description/></code> , <code><url/></code> , licenses, developers, scm)	Code Quality	Newly identified
L-11 NEW	<code>BackendApplicationTests.contextLoads()</code> is an empty no-op — always passes, tests nothing	Testing	Newly identified
L-12 NEW	<code>skill_front.md</code> documentation references deleted files and contains contradictions with current code	Documentation	Newly identified

Production Readiness Score

18

out of 100

Not production ready. This is pre-alpha / coursework-stage code. The one user-facing flow that exists (login) is functionally broken end-to-end — a successful login navigates nowhere and issues no credential. The system's actual business purpose (bookings, invoicing, penalties) has no API surface. All endpoints are publicly accessible with no authorization. Database credentials are hardcoded in version control. The path to MVP is clearly mapped — the architecture is sound; what's missing is implementation.

Dimension	Score	Key Rationale
Security	10/100	No token, all <code>permitAll()</code> , unauthenticated deletion, credentials in git, login leaks user existence
Authentication	5/100	BCrypt hashing only positive; no token, no session, no auth state anywhere
Authorization	0/100	No authorization model exists at any layer
Completeness	10/100	Login doesn't navigate; core domain (bookings) fully unimplemented as API
Backend Architecture	45/100	Clean layer separation in <code>auth/</code> ; no validation, no exception handler, no logging, wrong HTTP codes
Frontend Architecture	30/100	Clean view/component split; TS partially adopted; no proxy, no env vars, dead routes, no post-login navigation
Infrastructure	35/100	Docker mostly correct (DB port fixed); Dockerfile EXPOSE wrong; hardcoded secrets; no profiles; no migrations
Testing	40/100	Good unit coverage for domain classes; zero integration tests; context test is a no-op
Observability	10/100	No application logging; only TRACE SQL; no real health endpoint
Documentation	40/100	<code>skill_*.md</code> self-audits are detailed but now stale; project README is solid

Top 10 Remaining Issues

#	ID	Issue	Why It Comes First
1	C-01	No real authentication — no token issued, no navigation on login success	Every other security fix depends on having a real auth mechanism. The login flow is currently a no-op.
2	C-02	Unauthenticated user enumeration and arbitrary account deletion	Direct, trivially exploitable destructive endpoint with no auth required.
3	C-03	Hardcoded DB credentials in version control	Secret is already in git history; risk escalates with every fork or push.
4	H-08	No uniqueness constraint on <code>username</code> — duplicate accounts break login	Fundamental data-integrity defect that silently corrupts auth state.
5	H-01	Login error messages leak user existence	Quick to fix; amplifies C-02 damage even after that endpoint is secured.
6	H-03	No global exception handler — all errors return 500	Makes the API impossible to consume correctly; masks bad credentials as crashes.
7	H-04	No request validation — empty/blank credentials accepted	Allows degenerate accounts; enables future validation bypass without this layer.
8	M-05	No Vite proxy / no env vars — API URL unreachable in dev	Application literally cannot reach its own backend in the documented local dev workflow.
9	H-05 + H-06	Login succeeds with no navigation; <code>SecretView/SecretDisplay</code> are dead code	The only user-facing feature is a dead end; orphaned code doubles the surface area to understand.
10	M-03	<code>ddl-auto=update</code> with no migration tool; <code>Esquema_BD.sql</code> unenforced	Any future booking entity work will be built on uncontrolled schema drift.

Recommended Fix Order

The following order is designed to minimize technical risk at each step and ensure that downstream work is never built on a broken foundation. Each phase has a clear exit criterion before the next begins.

PHASE 1

Make the existing login feature actually work

*Exit: a user can log in, receive a token, and
navigate to a protected page*

1. **C-03** — Rotate and externalize the DB password before touching any other infrastructure. Do this in an isolated commit so the new secret is never committed to git.
2. **H-02** — Delete `com.novafacts.SecurityConfig` and `com.novafacts.config.PasswordConfig`. Zero risk; eliminates ambiguity before adding any new security code.
3. **H-07** — Remove `.httpBasic()` from `SecurityConfig`. Add `STATELESS` session policy. Establish the correct baseline before building JWT on top.
4. **C-01 (core)** — Implement JWT issuance in the backend (`LoginResponse` adds `token` ; add a `JwtFilter` ; protect at least `/api/users`). Largest single change — do it here, not at the end.
5. **H-01** — While implementing C-01, collapse the two distinct error messages into a single generic `"Credenciales inválidas"`. Zero extra effort if done simultaneously.
6. **H-05** — Import `useRouter` in `LoginView.vue`, store the token, call `router.push()`. One-line change that completes the login flow.
7. **M-05** — Add `server.proxy` to `vite.config.js` and introduce `.env` / `.env.production` files. Required for the frontend to actually reach the backend during development.

PHASE 2

Close security and correctness gaps

Exit: no unauthenticated destructive endpoints; all errors return correct HTTP codes; no blank accounts

1. **C-02** — Restrict `GET /api/users` and `DELETE /api/users/{id}` with `@PreAuthorize` or `requestMatchers().authenticated()`. Add `@ResponseStatus(HttpStatus.NO_CONTENT)` to `deleteUser()`.
2. **H-08** — Add `unique = true` to `User.username`.
3. **H-03** — Add `@ControllerAdvice` in `com.novafacts.backend.common`. Map exceptions to correct HTTP status codes.
4. **H-04** — Add `spring-boot-starter-validation`; annotate DTOs; add `@Valid` to controller parameters. Validation errors surface as 400s automatically via H-03.
5. **M-06** — Add router-level `beforeEach` guard. Now that Phase 1 exists, this is implementable against real auth state.
6. **H-06** — Delete `SecretView.vue`, `SecretDisplay.vue`, and the `/secret` route.
7. **M-08** — Add `@Slf4j` to `UserService`; log login attempts, user creation, and deletions at `INFO`.
8. **M-09** — Add correct `@ResponseStatus` annotations to `createUser()` and `deleteUser()`.

PHASE 3

Infrastructure and configuration hardening

Exit: system is safe to deploy to a staging environment

1. **M-03** — Adopt Flyway. Convert `Esquema_BD.sql` to the first migration (after reconciling `usuario` vs. `User`). Switch `ddl-auto` to `validate` or `none`.
2. **M-02** — Split `application.properties`; move verbose SQL logging to `application-dev.properties`. Add `SPRING_PROFILES_ACTIVE=prod` to `docker-compose.yml`.
3. **M-04** — Fix `EXPOSE 8081` → `EXPOSE 8082` in Dockerfile.
4. **M-07** — Add pagination (`Pageable`) to `getUsers()`.
5. **L-11** — Convert `BackendApplicationTests` to a real context test with `Testcontainers`, or `@Disabled` it explicitly.

PHASE
4**Code quality and developer
experience cleanup***Exit: no dead code; consistent JS/TS; clean
repository structure*

1. **L-01** — Delete `TestController` and `HelloController`. Add Spring Boot Actuator if health checks are needed.
2. **L-02** — Delete `HelloWorld.vue`, `vite.svg`, `vue.svg`, `hero.png`, `style.css`.
3. **L-04 + L-05** — Delete orphaned `package-lock.json` files and stale `backend/` workspace directory.
4. **L-09** — Add `tsconfig.json`; convert `main.js`, `router/index.js` to TypeScript; add `vue-tsc` to the build script.
5. **L-06 + L-07 + L-08** — Remove the dead forgot-password link, the fake email validation, and implement or remove "remember me".
6. **L-10** — Populate `pom.xml` metadata fields.
7. **L-12** — Update `skill_front.md` and `skill_back.md` to reflect post-Phase-2 state.

PHASE
5**Domain implementation
— the actual product***Exit: bookings, invoicing, and financial flows
reachable over HTTP and persisted correctly*

1. **C-04** — Design and implement the booking domain layer: JPA entities for `reserva`, `factura`, `anticipo`, `penalidad`, `nota_credito`, `devolucion`; repositories; services; REST controllers.
2. **M-01** — Replace `double` with `BigDecimal` in `Booking`, `InvoiceCalculator`, and all persistence-layer monetary fields as part of the domain implementation.